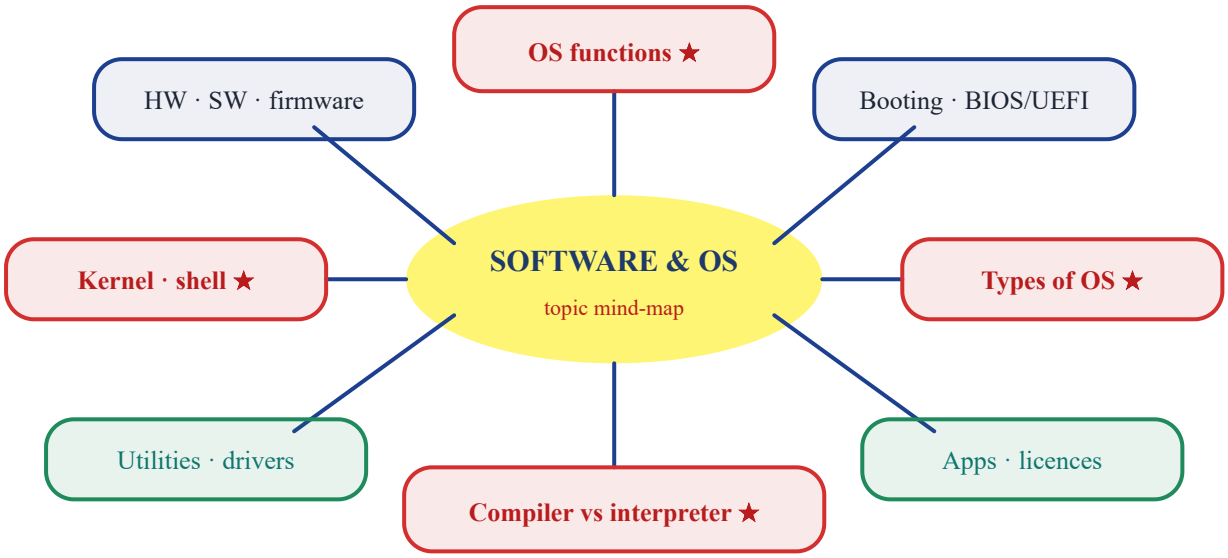
Software & Operating System

System vs application · OS · kernel · translators · licences: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = applied

Syllabus coverage

"Introduction to Computer Systems: … Major components of a computer; Hardware: processor, memory, I/O devices; **Software: Operating system, application software**; Basic architecture of a computer …". This notebook covers the software half: classification, the OS (role, functions, kernel, types, booting), translators, utilities, drivers, firmware, application software and licences. Hardware and architecture are in the **Hardware Components** notes (cs2); processes, scheduling, memory management and deadlock in depth are in **Chapter 18: Operating System**.

High-yield list

① system vs application
② kernel vs shell
③ compiler vs interpreter
④ multiprogramming vs time-sharing vs multiprocessing
⑤ hard vs soft real-time
⑥ freeware · shareware · open source

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

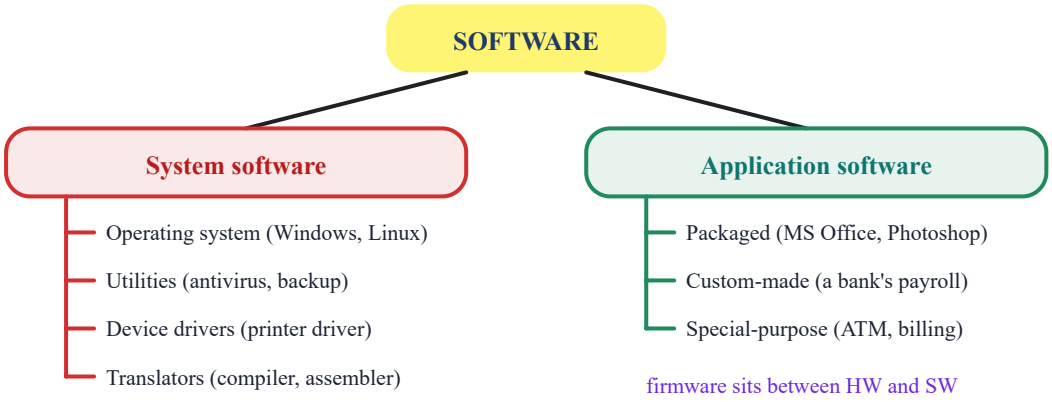
✓ Software & its classification …… p. 2
✓ OS: role, functions, booting …… p. 3
✓ Kernel, shell, system calls, UI …… p. 4
✓ Types of operating systems …… p. 5
✓ Popular OS, history, file systems … p. 6
✓ Language translators …… p. 7
✓ Utilities, drivers, firmware …… p. 8
✓ Application software & licences … p. 9
✓ Night-before sheet …… p. 10
✓ Practice MCQs + written …… p. 11

1 What Is Software?

Definition: **Software** is a set of **programs**, with their data and documentation, that tells the hardware what to do. It is **intangible**: it does not wear out, but it can have **bugs** and become obsolete. The word "software" was first used in print by John W. Tukey (**1958**).

Component	What it is	Examples
Hardware	physical, tangible parts (see cs2)	CPU, RAM, keyboard, SSD
Software	programs + data, intangible	Windows, MS Word, Chrome
Firmware	software fixed in ROM/flash of a device	BIOS/UEFI, router firmware
Humanware (liveware)	people who use and run the system	users, programmers, operators

2 Classification of Software



system software serves the machine; application software serves the user

Point	System software	Application software
Purpose	runs the computer; a **platform** for apps	does one task for the user
Runs	from boot to shutdown, in the background	only when the user opens it
Dependence	needed before any app can run	runs **on top of** the OS
Written in	C, C++, assembly (close to hardware)	any high-level language

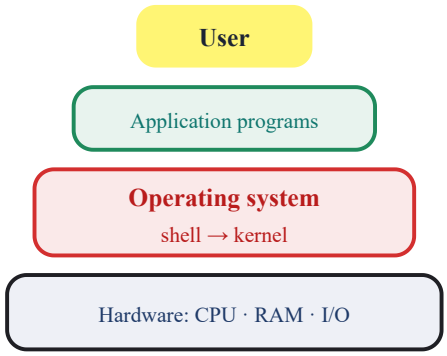
Memory trick: System software = "**OUT-D**": **O**S, **U**tilities, **T**ranslators, **D**rivers. Whatever the user opens to get work done is an application.

Exam trap: Antivirus (utility), compiler (translator) and device driver are **system** software. A web browser and a word processor are **application** software.

Worked example ✍️: Windows 11 → system (OS) · Microsoft Defender → system (utility) · GCC → system (translator) · Excel → application (packaged) · BIOS → **firmware**

3 Operating System: Role & Functions

Definition: The **operating system (OS)** is the system software that **manages the hardware resources** and gives users and programs an **interface** and common services. Two views: a **resource manager** (shares out CPU, memory, devices, files) and an **extended machine** (hides hardware detail behind simple calls). It is the first major program loaded at boot; its kernel stays in RAM until shutdown.



requests go down, results come up

Function	What the OS does
Process	create, schedule, end processes; **process = a program in execution** (Ch 18)
Memory	allocate/free RAM; **virtual memory** = disk as RAM (pagefile.sys, swap)
File	create, read, write, delete files; directories; permissions
Device (I/O)	drivers, interrupts, buffering, **spooling** (print queue)
Storage	free-space tracking, disk scheduling, formatting
Security	login, user accounts, access rights; isolates programs
User interface	CLI or GUI (sheet 4)
Others	networking, error detection, accounting (usage logs)

Memory trick: "**P**lease **M**ake **F**riendly **D**evices **S**afe & **U**seful" → Process, Memory, File, Device, Security, User interface.

SPOOL: = **S**imultaneous **P**eripheral **O**perations **On-L**ine: jobs wait on disk in a queue so a slow printer never holds up the CPU.

Booting: from power-on to login

1. Power on → **BIOS/UEFI** (firmware in ROM/flash) runs the **POST** (Power-On Self-Test)
2. It finds the boot device (boot order) and loads the **bootloader** from the **MBR** (first 512-byte sector) or the EFI partition (GPT)
3. The bootloader (GRUB, Windows Boot Manager) loads the **kernel** into RAM
4. Kernel starts drivers and the first process (init/systemd) → services → login

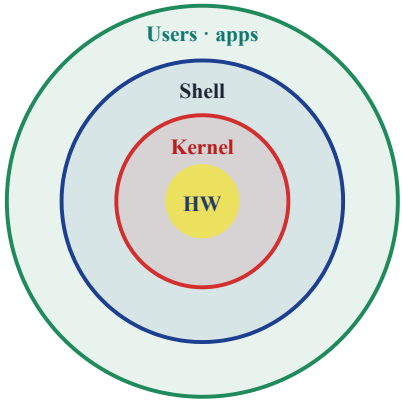
BIOS = Basic Input/Output System; UEFI = Unified Extensible Firmware Interface

Exam trap: **Cold boot** = start from power-off; **warm boot** = restart without cutting power (Restart, Ctrl+Alt+Del). The bootstrap loader lives in **ROM/flash**, not on the hard disk; BIOS settings are kept in battery-backed **CMOS** RAM.

4 Kernel, Shell & System Calls

Kernel: The **core of the OS**: always resident in main memory, runs in privileged **kernel mode**, manages CPU, memory, devices and files.

Shell: The **command interpreter**: the outer layer that takes the user's commands and passes them to the kernel. Text shells: sh (Bourne), **bash** (default on most Linux), zsh (macOS default since 2019), cmd.exe, PowerShell. GUI shell: Windows Explorer.



the shell wraps the kernel

Memory trick: Like a nut: the **kernel** is the seed inside, the **shell** is the cover outside. You talk to the shell; the shell talks to the kernel.

Kernel type	Idea	Examples
Monolithic	all services in kernel space; fast	**Linux**, UNIX, MS-DOS
Microkernel	tiny kernel; drivers, file system in user space	MINIX, QNX, L4
Hybrid	micro design + some services in kernel	Windows NT, macOS (XNU)

User mode vs kernel mode: apps run in user mode with limited rights. To use hardware they make a **system call**: a trap (software interrupt) switches the CPU to kernel mode, the kernel does the job, then control returns to user mode.

```
fd = open("a.txt", O_RDONLY);
read(fd, buf, 100); /* kernel */
close(fd);   /* back in user mode */
```

UNIX/Linux calls: fork(), exec(), open(), read(), write(), exit() · Windows: CreateProcess(), ReadFile() · libraries (POSIX, Win32 API) wrap them.

User interface: CLI vs GUI

Point	CLI (Command Line)	GUI (Graphical)
Interaction	typed commands (dir, ls, cd)	**WIMP**: windows, icons, menus, pointer
Resources	low; ideal for servers, scripts	more RAM, CPU, graphics
Learning	harder; fast for experts	easy for beginners
Examples	MS-DOS, Linux terminal	Windows, macOS, Android

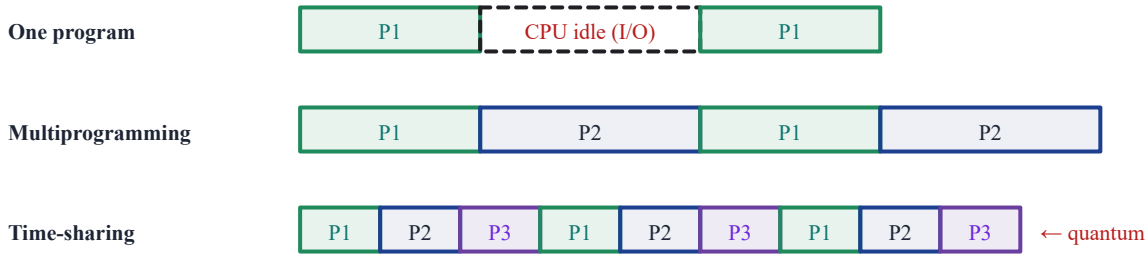
GUI history: Xerox **Alto** (PARC, **1973**) → Apple Lisa (1983) → Macintosh (1984) → Windows 1.0 (1985).

Exam trap: The shell is **not** the kernel, and a "terminal" is only the window that runs a shell. MS-DOS is a CLI OS; Windows 1.0–3.x were GUIs running on top of DOS.

5 Types of Operating Systems

Type	Key idea	Examples / use
Batch	similar jobs grouped and run in sequence; no user interaction	early IBM mainframes
Multiprogramming	many programs in RAM; CPU switches when one waits for I/O → **CPU utilisation**	IBM OS/360
Time-sharing (multitasking)	CPU time cut into slices (quantum) and rotated → **fast response**	CTSS (1961), Multics, UNIX
Multiprocessing	2+ CPUs/cores share memory, run in **parallel**; SMP or AMP	multi-core PCs, servers
Multithreading	one process split into concurrent threads	browsers, web servers
Real-time (RTOS)	output within a deadline; **hard**: a miss = failure; **soft**: a miss is tolerated	VxWorks, QNX, FreeRTOS
Distributed	networked computers appear to the user as **one** system	Amoeba, clusters, cloud
Network OS	a server shares files, printers, users over a LAN	NetWare, Windows Server
Embedded / mobile	built into devices; small footprint; phones	embedded Linux, Android

Hard real-time: airbag, pacemaker, flight control. Soft real-time: video streaming, VoIP, online games.



one CPU: multiprogramming fills the I/O gaps; time-sharing rotates fixed slices

By users & tasks:

Single-user, single-task: **MS-DOS**

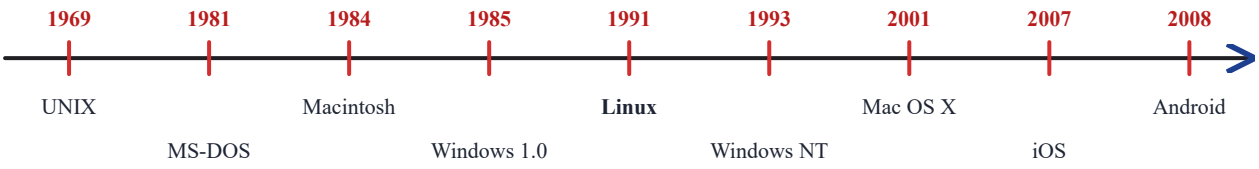
Single-user, multitasking: Windows 11, macOS

Multi-user: **UNIX**, Linux, Windows Server

Memory trick: Multi-**programm**ing = many **programs** in RAM · multi-**task**ing = time **slices** · multi-**process**ing = many **processors**.

Exam trap: Goal of multiprogramming = **CPU utilisation**; goal of time-sharing = **response time**.
Multitasking works on one CPU (concurrency); multiprocessing needs more than one CPU (true parallelism).

6 Popular Operating Systems & Milestones



OS milestones (Android: first phone, HTC Dream, 2008)

OS	Origin	Key facts
UNIX	AT&T **Bell Labs**, 1969: Ken Thompson & Dennis Ritchie	rewritten in **C** in 1973 → portable; multi-user, multitasking; BSD family
MS-DOS	Microsoft, 1981 (from 86-DOS) for the IBM PC	single-user, single-task, CLI; 16-bit
Windows	Microsoft; 1.0 in 1985; NT line from 1993	proprietary; hybrid NT kernel; most-used desktop OS
Linux	**Linus Torvalds, 1991** (Finland)	monolithic kernel, open source under **GPL**; mascot Tux; distros: Ubuntu, Debian, Fedora, Red Hat, Mint
macOS	Apple; Mac OS X in 2001	XNU hybrid kernel on Darwin (BSD-based); certified UNIX, **not** Linux
Android	Android Inc. (2003), bought by **Google** in 2005	built on the **Linux kernel**; apps in .apk; open source (AOSP)
iOS	Apple, 2007 (first iPhone)	proprietary; XNU kernel like macOS
ChromeOS	Google, 2011 (Chromebook)	Linux kernel; web/cloud-centred

GNU project: Richard Stallman, **1983** (a free UNIX-like OS). Its tools + the Linux kernel make a complete free OS.

> "Linux" = only the kernel; the full OS = GNU/Linux

File systems (managed by the OS)

File system	Used by	Remember
FAT32	old Windows, USB drives	max file size **4 GB**; no permissions
exFAT	flash drives, SD cards	large files, cross-platform
NTFS	Windows NT/XP/10/11	journaling, permissions, encryption, compression
ext4	Linux	journaling; default on most distros
APFS / HFS+	macOS, iOS	APFS since 2017; HFS+ before

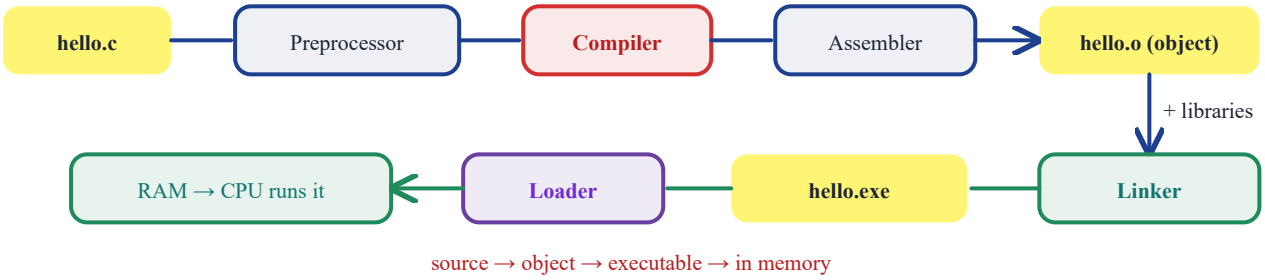
Memory trick: **"69 UNIX, 81 DOS, 85 Windows, 91 Linux"**: say it as a rhyme of four years.

Exam trap: Android uses the Linux kernel; macOS and iOS are UNIX-based but **not** Linux; Windows is neither.

7 Language Translators

Definition: A **language translator** is system software that converts a program in one language (**source code**) into machine code (**object code**) the CPU can run. Three kinds: **assembler**, **compiler**, **interpreter**.

Gen.	Language level	Example	Translated by
1GL	binary machine code; fastest	10110000	nothing
2GL	assembly: **mnemonics**	MOV AL, 61h	**assembler**
3GL	high-level, procedural	C, Java, Python	compiler / interpreter
4GL	non-procedural: say **what**, not how	SQL	DBMS engine
5GL	logic / constraints (AI)	Prolog	inference engine



Point	Compiler	Interpreter
Translates	**whole program at once**, before running	one statement at a time
Output	separate object / executable file	no object file
Errors	listed after the whole program is scanned	stops at the **first** error
Speed	program runs **faster**	slower (translates every run)
Re-run	no compiler needed	interpreter needed every time
Examples	C, C++, Fortran, Go	Python, BASIC, JavaScript, PHP

Linker: joins object files and library code into one **executable**. **Loader**: copies the executable into RAM and starts it.

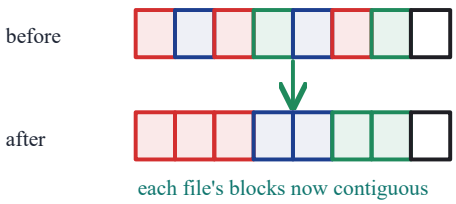
Memory trick: Compiler = translate the **whole book**, then read it. Interpreter = a **live interpreter** at a meeting, sentence by sentence.

Exam trap: **Java** is both: javac compiles to **bytecode** (.class) and the JVM interprets or JIT-compiles it. An assembler handles **assembly only**; object code cannot run until it is linked.

8 Utility Software

Definition: **Utility software** is system software that **maintains, analyses, optimises and protects** the computer: the housekeeping tools of the OS.

Utility	Job	Examples
Antivirus	detects and removes malware	Defender, Kaspersky
Defragmenter	joins a file's scattered blocks	"Optimize Drives"
Disk cleanup	removes temporary and junk files	Disk Cleanup
Backup	copies data so it can be restored	Time Machine (macOS)
Compression	shrinks files (.zip, .rar, .7z)	WinZip, WinRAR, 7-Zip
Format / partition	prepares and divides a disk	Disk Management, fdisk
File manager	browse, copy, move, delete files	File Explorer, Finder
System monitor	shows CPU, RAM, processes	Task Manager, top



each file's blocks now contiguous

Exam trap: Defragmenting helps a **hard disk** (moving head). An **SSD** has no moving parts, so it is not defragmented; the OS sends TRIM instead. Task Manager: **Ctrl+Shift+Esc**.

9 Device Drivers & Firmware

Device driver: A system program that lets the OS **talk to one specific device**: it turns general I/O requests into that device's own commands. Usually runs in kernel mode (.sys files in Windows). **Plug and Play**: the OS detects a new device and loads its driver.

Firmware: Software **stored permanently** in non-volatile memory (ROM, EEPROM, flash) inside hardware: BIOS/UEFI, routers, printers, SSD controllers. Updated by "flashing".



"Print" goes through the OS and the driver to the device

Point	Legacy BIOS	UEFI
Partition table	MBR: disks up to **2 TB**, 4 primary	GPT: bigger disks, 128 partitions
Features	16-bit, text-only setup	32/64-bit, Secure Boot, mouse

Memory trick: **Utility** keeps the PC healthy · **driver** lets a device talk · **firmware** lives inside the chip.

10 Application Software

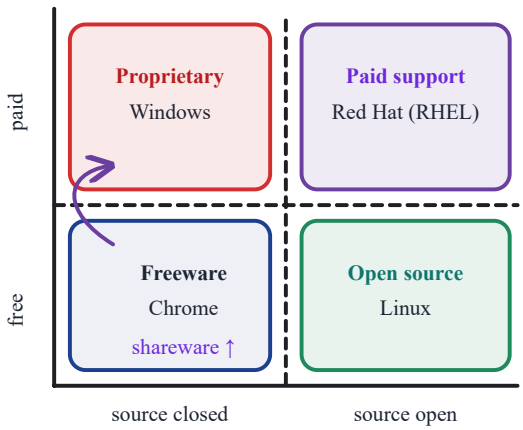
Definition: Software that does a **specific task for the user** using the services of the OS. **Packaged / off-the-shelf**: general-purpose, sold to many (MS Office) · **custom / tailor-made**: built for one organisation (a bank's core banking system) · **special-purpose**: one job (ATM, ticketing). A **suite** bundles related programs (MS Office, LibreOffice).

Category	Examples
Word processor	MS Word, Google Docs, LibreOffice Writer
Spreadsheet	Excel, Calc; first one: **VisiCalc** (1979)
Presentation · database	PowerPoint, Impress · MS Access, MySQL, Oracle
Desktop publishing (DTP)	Adobe InDesign, PageMaker, QuarkXPress
Graphics: raster · vector	Photoshop, GIMP · Illustrator, CorelDRAW
Multimedia · browser	VLC, Premiere Pro · Chrome, Firefox (cs4)
Enterprise · accounting · CAD	SAP ERP · Tally, QuickBooks · AutoCAD
Scientific · statistics	MATLAB · SPSS
Bangla typing	**Bijoy** (Mustafa Jabbar) · **Avro** (open source)

11 Software Licences

Licence	Meaning	Examples
Proprietary	owned; source hidden; licence fee (EULA)	Windows, MS Office
Freeware	**free of cost**, but source closed	Acrobat Reader, Chrome
Shareware	**try before you buy**: trial, then pay	WinRAR, WinZip
Open source	source available; study, modify, share	Linux, Firefox, LibreOffice
Public domain	no copyright at all	SQLite

Free software (Richard Stallman; GNU 1983, **FSF 1985**) is "free as in **freedom**, not free beer": the four freedoms to run, study, share and modify. **GPL** is **copyleft**: modified versions must stay under the GPL. MIT, BSD and Apache licences are **permissive**. "Open source" as a term: OSI, 1998.



> **Memory trick:** **Freeware** = free beer (no price) · **free software** = free speech (freedom) · **shareware** = taste first, then pay.

> **Exam trap:** Freeware ≠ open source: Chrome is freeware, Chromium is open source. Open-source software may still be **sold** (Red Hat sells subscriptions and support).

Night-Before-the-Exam Sheet

Fact sheet:

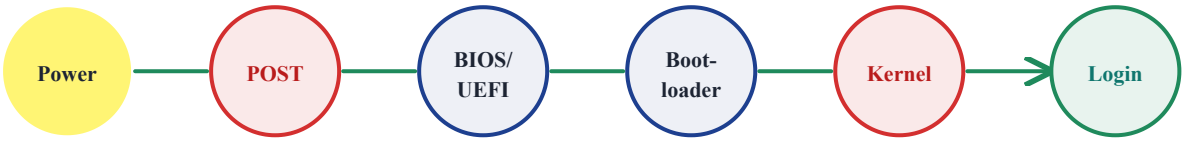
Software = programs + data + docs	System SW = OS, utilities, translators, drivers
OS = resource manager + extended machine	Process = program in execution
Kernel = core, always in RAM	Shell = command interpreter
System call: user mode → kernel mode	Linux monolithic · MINIX micro · Win hybrid
Multiprogramming → CPU utilisation	Time-sharing → response time
Hard RTOS: missed deadline = failure	POST by BIOS/UEFI; MBR = 512 bytes
Compiler: whole program, faster run	Interpreter: line by line, stops at 1st error
Assembler: assembly → machine code	Linker joins · loader loads into RAM
FAT32 file limit 4 GB · NTFS · ext4	Freeware free · shareware trial · GPL copyleft

Top 12 one-liners for MCQs

1. UNIX: Bell Labs, 1969 (Thompson, Ritchie)
2. Linux kernel: Linus Torvalds, 1991
3. MS-DOS: 1981, single-user, CLI
4. Android: Linux kernel; Google, 2005
5. First GUI: Xerox Alto, 1973
6. SPOOL: Simultaneous Peripheral Operations On-Line
7. Java: bytecode run by the JVM
8. 4GL example: SQL; 5GL: Prolog
9. BIOS is firmware; settings in CMOS
10. GNU: Stallman, 1983; FSF, 1985
11. First spreadsheet: VisiCalc, 1979
12. "Bug": moth in Harvard Mark II, 1947

Top traps: antivirus, compiler, driver = system software · browser = application · shell ≠ kernel · multitasking needs 1 CPU, multiprocessing needs 2+ · macOS is UNIX, not Linux · assembler ≠ compiler · freeware ≠ open source · SSDs are not defragmented · warm boot = restart

All tricks together: OUT-D = OS, Utilities, Translators, Drivers · "Please Make Friendly Devices Safe & Useful" = OS functions · nut: kernel inside, shell outside · program-ming / task-ing / process-ing · 69-81-85-91 = UNIX, DOS, Windows, Linux · whole book vs live interpreter · utility heals, driver talks, firmware lives in the chip · free beer vs free speech



MBR (512 bytes) or GPT/EFI partition → GRUB / Windows Boot Manager

the boot chain in one line

✓ Practice MCQs (exam pattern, 10 marks)

1. Which of these is NOT system software?
(a) Device driver (b) Compiler
(c) MS Excel (d) Defragmenter

2. The core of the OS that always stays in main memory:
(a) Shell (b) Kernel
(c) BIOS (d) Loader

3. The Linux kernel was first released by:
(a) D. Ritchie (b) R. Stallman
(c) Bill Gates (d) L. Torvalds

4. Which translator executes a program line by line?
(a) Interpreter (b) Compiler
(c) Assembler (d) Linker

5. An OS in which a missed deadline is a system failure:
(a) Batch (b) Time-sharing
(c) Hard real-time (d) Network

6. In SPOOL, the letter S stands for:
(a) Serial (b) System
(c) Shared (d) Simultaneous

7. Software stored permanently in ROM or flash:
(a) Shareware (b) Firmware
(c) Middleware (d) Freeware

8. Free for a trial period, then must be paid for:
(a) Freeware (b) Open source
(c) Shareware (d) Public domain

9. Combines object files and libraries into an executable:
(a) Loader (b) Assembler
(c) Debugger (d) Linker

10. The main goal of multiprogramming is higher:
(a) CPU utilisation (b) disk capacity
(c) screen resolution (d) network speed

Answer key → 1-c · 2-b · 3-d · 4-a · 5-c · 6-d · 7-b · 8-c · 9-d · 10-a

Written question: What is an operating system? Describe five of its functions and distinguish between multiprogramming and time-sharing. 8 marks

1. **Definition:** system software that manages hardware resources and gives users and programs an interface and common services; e.g. Windows, Linux. 1
2. **Process:** create, schedule, end · **Memory:** allocate RAM, virtual memory · **File:** organise, protect files · **Device:** I/O via drivers, spooling · **Security:** login, rights. 5

3 **Multiprogramming** keeps several programs in RAM and switches the CPU when one waits for I/O → aim: **CPU utilisation**. **Time-sharing** gives each process a fixed time slice in turn → aim: **fast response** for interactive users. 2

Before the exam ✓

- ✓ Classify any program: system or application
- ✓ List OS functions and the boot steps
- ✓ Kernel vs shell, CLI vs GUI
- ✓ Translators, OS years, licence types

Also practise (written):

① Compiler vs interpreter (5 points)
② Types of OS with examples
③ System vs application software
④ Booting process of a PC